

The next 10 years will be focused on data.

Programming languages will evolve so as to facilitate processing of data to obtain actionable intelligence. If Fortran drove the development of high-performance scientific computing, and 'C' drove the development of complex system software like UNIX, the next few years will see languages like R, MATLAB and their successors evolve to process and prune data, and present the user with 'intelligence' from the data. The last 10 years drove programming languages to evolve towards the Web 2.0 revolution. The next 10 years will be about the evolution of programming languages for 'Data Science'.

My 'must-read book' for this month

This month's 'must-read' suggestion comes from me. It is not for a book, but an article that I strongly believe needs to be read by each aspiring programmer. It is titled 'Teach Yourself Programming in 10 Years' by Peter Norvig. For those who don't know, Norvig is the director of research at Google, and a pioneer in the field of artificial intelligence. (On a separate note, his online class on artificial intelligence available in <http://www.udacity.com> is a great course to take.) Norvig presents his recipe on how to be successful in programming.

As we all know, there is no easy short-cut to becoming a successful programmer—no matter how many books you read that promise to teach you to be a C, C++ or Java programmer in three or 21 days. The article is available at <http://norvig.com/21-days.html>.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful, so I can mention it in the column. This would help many readers who want to improve their coding skills.

If you have any favourite programming puzzles that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at sandyasm_AT_yahoo_DOT_com. Till we meet again next month, happy programming and here's wishing you the very best! 

By: Sandya Mannarswamy

The author is an expert in systems software and is currently in a happy state between jobs. Her interests include compilers, multi-core technologies and software development tools. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182>.

Continued from page no 46...



The sinking of the Vasa

This sad, yet amusing, story may strike a chord with many software developers! Unfortunately, most software projects are strikingly similar to the story of how the Vasa was constructed. For example, requirements are volatile, and feature requests keep coming in. People involved in the projects are volatile as well, with key developers moving in and out. Aspects such as specifications and documentation are not given the necessary importance, and are considered a waste of time, especially under the guise of following an Agile/Lean approach. Testing is not automated, and the

'big-bang' method of integration or systems testing is the norm. Finally, let's not ignore the unethical practice of hiding from the 'king' the stability problems found during testing and not sharing the obvious fact that the ship is likely to sink.

Of all these, I would consider the lack of a disciplined approach to be the

major cause of software failure – a factor that still plagues the industry. Yes, of course, agility is important. Hence, what software projects need to succeed is the right balance between agility and discipline, and a culture that learns from mistakes. 

By: S G Ganesh

The author works for Siemens (Corporate Research & Technologies), Bangalore. You can reach him at sgganesh@gmail.com.